

DevOps Case Study Report

Candidate scope: Task 0 Kubernetes Deployment, Task 1 Jenkins CI/CD Pipeline, Task 3 Monitoring & Observability.

1. Architecture

The demo repository contains a Python FastAPI service, a Jenkins pipeline, Helm charts for the application and monitoring stack, and a lightweight Prometheus/Grafana deployment for a local Kubernetes cluster. The application exposes `/`, `/health`, `/ready`, `/work`, and `/metrics`. Prometheus scrapes application metrics through the in-cluster Kubernetes service `demo-app:80`, and Grafana provisions a dashboard for request rate and latency.

Local components:

- `demo-app`: FastAPI application.
- `jenkins`: Jenkins controller with Docker, Python, kubectl, and pipeline plugins.
- `jenkins-agent`: optional inbound agent image.
- `docker-registry`: local registry for image push demos.
- `monitoring` Helm release: Prometheus and Grafana running inside Kubernetes.

Some credentials are intentionally mocked or created manually in Jenkins because the submitted repository must not contain secrets.

2. Pipeline Design

The `Jenkinsfile` is a multi-stage pipeline:

1. Checkout source code.
2. Validate and Test in parallel:
 - Python unit tests with JUnit report publishing.
 - Helm chart linting and manifest rendering.
3. Docker Build with BuildKit and immutable image tag.
4. Image Push to local registry or authenticated external registry.
5. Load the image into kind for local demos when enabled.
6. Deploy to Kubernetes with `helm upgrade --install`, image overrides, environment overrides, `--wait`, and rollout status verification.
7. Deploy Prometheus and Grafana to Kubernetes with the `helm/monitoring` chart when enabled.

Error handling is implemented with explicit rollout checks and `post { failure { ... } }`, which collects Kubernetes deployment, service, HPA, pod, and Helm release history to speed up troubleshooting. The pipeline disables concurrent builds to avoid two deployments racing on the same namespace.

Image versioning uses:

```
BUILD_NUMBER-GIT_SHA_SHORT
```

This makes each deployment traceable while still pushing `latest` for local demo convenience.

For local kind demos, the pipeline can run `kind load docker-image` after image push. This keeps the required registry push stage while avoiding the common kind issue where pods cannot pull `localhost:5000` from inside the Kubernetes node.

3. Kubernetes Deployment

The primary Kubernetes deployment path uses the Helm chart in `helm/demo-app/`:

- `values.yaml`: default image, environment, replicas, resources, probes, service, and HPA settings.
- `templates/deployment.yaml`: two replicas, rolling update strategy, probes, resource requests and limits.
- `templates/service.yaml`: internal ClusterIP service.
- `templates/configmap.yaml`: environment-specific non-secret configuration.
- `templates/hpa.yaml`: optional HorizontalPodAutoscaler.

The deployment uses a RollingUpdate strategy with `maxUnavailable: 0` and `maxSurge: 1` so a new pod must become ready before old capacity is removed. Readiness and liveness probes reduce the chance of routing traffic to unhealthy pods.

This demo uses Helm in the Jenkins deployment stage because the pipeline needs repeatable releases, per-environment overrides, release history, and rollback support. `kubectl apply` is still useful for small manual tests, but Helm is a better fit for CI/CD because one command can install or upgrade the release while preserving revision history for rollback.

4. Rollback and Scaling

The app uses a Kubernetes RollingUpdate strategy with readiness probes. If a new deployment is unhealthy, Kubernetes does not route traffic to pods that are not ready. Rollback can be done manually after inspecting the failure.

Rollback with Helm:

```
helm history demo-app
helm rollback demo-app <revision>
kubectl rollout status deployment/demo-app
```

Rollback with Kubernetes deployment history:

```
kubectl rollout history deployment/demo-app
kubectl rollout undo deployment/demo-app
```

In Jenkins, `helm upgrade --install` runs with `--wait --timeout 120s`, then the pipeline checks `kubectl rollout status`. If the deployment fails, the failure block prints Helm history and Kubernetes diagnostics so the operator can roll back to the previous known-good revision.

Scaling can be done manually:

```
kubectl scale deployment/demo-app --replicas=4
```

Or automatically through the Helm chart HPA template, which targets 70 percent CPU utilization and scales between two and five replicas.

5. Monitoring and Observability

The app exposes Prometheus metrics via `/metrics`. The monitoring stack is deployed into the same kind cluster with the `helm/monitoring` chart:

- `prometheus`: scrapes `demo-app:80/metrics` inside the cluster.
- `prometheus`: scrapes Jenkins at `/prometheus` for pipeline metrics.

- ``node-exporter``: exposes node CPU, memory, filesystem, and host-level metrics.
- ``grafana``: uses Prometheus as a provisioned datasource.
- ``grafana-dashboard-demo-app``: provisions the demo dashboard from a ConfigMap.

Application metrics:

- ``demo_app_http_requests_total``
- ``demo_app_http_request_duration_seconds``

Jenkins metrics are exposed by the Jenkins Prometheus plugin. System metrics are exposed by node-exporter.

Important metrics:

- Request rate by path.
- HTTP 5xx rate.
- p95/p99 latency.
- Pod CPU and memory usage.
- Kubernetes restart count.
- Jenkins build duration and failure rate.
- Jenkins scrape health and last build duration.
- Node CPU and memory usage from node-exporter.

The demo dashboard includes application request rate and p95 latency, Jenkins scrape health and last build duration, and system CPU/memory usage from node-exporter. Jenkins console logs remain the source of detailed stage-by-stage troubleshooting.

Alert examples:

- p95 latency above 1 second for 5 minutes.
- 5xx error rate above 2 percent for 5 minutes.
- Pod restarts greater than 3 in 10 minutes.
- Jenkins pipeline failure on the main branch.

Production debugging flow:

1. Check Grafana for latency, error rate, and saturation changes.
2. Inspect recent deployment revision and image tag.
3. Check pod logs and Kubernetes events.
4. Compare CPU/memory usage against resource limits.
5. Roll back if user impact is high, then investigate root cause.

6. Troubleshooting

Common Kubernetes deployment issues:

- ``ImagePullBackOff``: wrong image tag, registry authentication failure, or local cluster cannot access local registry.
- Helm ownership conflict: resources were previously created by ``kubectl apply``; delete or migrate them before installing the Helm release.
- ``CrashLoopBackOff``: application startup error, missing environment variable, or invalid command.

- Readiness probe failure: wrong path, wrong port, slow startup, or dependency unavailable.
- Pending pods: insufficient CPU/memory or node scheduling constraints.
- Service unreachable: selector labels do not match pod labels or wrong target port.

Useful commands:

```
kubectl get pods,svc,deploy
kubectl describe pod <pod>
kubectl logs deploy/demo-app
kubectl rollout status deployment/demo-app
kubectl get events --sort-by=.lastTimestamp
```

7. Run Demo

Deploy to Kubernetes manually with Helm (app and monitoring)

Create a local kind cluster if needed:

```
kind create cluster --name devops-demo
kubectl get nodes
```

Build and load the app image into kind:

```
docker build -t devops-case-study/demo-app:local ./app
kind load docker-image devops-case-study/demo-app:local --name devops-demo
```

Deploy the app with Helm:

```
helm upgrade --install demo-app helm/demo-app \
--set image.repository=devops-case-study/demo-app \
--set image.tag=local \
--set app.env=dev \
--set app.version=local \
--wait \
--timeout 120s
kubectl rollout status deployment/demo-app
```

Deploy Prometheus and Grafana into the same cluster:

```
docker pull prom/prometheus:v2.54.1
docker pull grafana/grafana:11.2.0
docker pull prom/node-exporter:v1.8.2
kind load docker-image prom/prometheus:v2.54.1 --name devops-demo
kind load docker-image grafana/grafana:11.2.0 --name devops-demo
kind load docker-image prom/node-exporter:v1.8.2 --name devops-demo
helm upgrade --install monitoring helm/monitoring \
--wait \
--timeout 180s
kubectl rollout status deployment/prometheus
kubectl rollout status deployment/grafana
```

Open services:

```
kubectl port-forward svc/demo-app 8080:80
kubectl port-forward svc/grafana 3000:3000
kubectl port-forward svc/prometheus 9090:9090
```

Deploy to Kubernetes automatically (use CI/CD of Jenkins)

Start Jenkins and the local Docker registry:

```
docker compose up --build -d jenkins docker-registry
```

Create a Jenkins-specific kubeconfig for kind and upload it as a secret file credential with ID `kubeconfig`:

```
cp ~/.kube/config ./kubeconfig-jenkins
sed -i 's#server: https://127.0.0.1:[0-9]*#server: https://devops-demo-control-
plane:6443#' ./kubeconfig-jenkins
```

Jenkins requires these credentials:

- `docker-registry-credentials` for external registry push.
- `kubeconfig` as a secret file for Kubernetes deployment.

Create a Pipeline job from SCM using the public GitHub repository and `Jenkinsfile`. Run `Build with Parameters`:

- `IMAGE\_REPOSITORY=localhost:5000/demo-app`
- `KUBE\_NAMESPACE=default`
- `KIND\_CLUSTER\_NAME=devops-demo`
- `LOAD\_IMAGE\_TO\_KIND=true`
- `DEPLOY\_MONITORING=true`
- `DEPLOY\_ENV=dev`

The Jenkins pipeline checks out code, runs tests, validates Helm charts, builds and pushes the Docker image, loads it into kind, deploys the app with Helm, and deploys monitoring with Helm. RollingUpdate and readiness probes protect traffic during deployment, and rollback is performed with Helm or Kubernetes rollout history if the release is unhealthy.

8. Design Decisions

FastAPI was selected because it is small, easy to test, and can expose metrics without extra infrastructure. Helm was selected for CI/CD deployment because it provides release history, value overrides, and repeatable install/upgrade behavior. Docker Compose runs Jenkins and the local registry, while Helm deploys the app, Prometheus, and Grafana into the kind cluster.

The repository keeps secrets out of source control. Jenkins credentials and kubeconfig are injected at runtime. This mirrors production CI/CD behavior while keeping the public repository safe.